Bit-Exact Self-Speculative Decoding for a Hybrid Linear-Attention LLM on a Bandwidth-Bound Laptop GPU

Venkata Sai Krishna Gani
Department of Computer Sciences, University of Wisconsin–Madison
ganisai8202@gmail.com

November 3, 2026

Abstract

Single-stream LLM decoding on laptops is limited by memory bandwidth. Speculative decoding should help, because verifying several drafted tokens in one forward pass reads the weights only once. We show that on an Apple M1 Pro this premise fails for 4-bit models in both MLX and llama.cpp: verifying T tokens costs roughly T single-token steps. We build a decoder for Qwen3.5-9B, a hybrid of Gated DeltaNet linear-attention and gated-attention layers, that makes speculation pay on this hardware. It combines self-speculation with the model's native multi-token-prediction head and exact rollback of the recurrent state; a multi-token 4-bit mat-vec kernel that verifies two tokens for 1.2–1.5× the cost of one; a *bit-exact* construction of that kernel, which reproduces the single-token kernel's floating-point summation order so that speculative output is byte-identical to plain greedy decoding; a trimmed-vocabulary draft head; optimistic pipelining; and, because each dependent kernel costs about 10 µs of latency on this GPU, one fused bit-exact kernel for each DeltaNet layer. On a 20-prompt suite, mean decoding speed of Qwen3.5-9B rises from 23.1 tok/s (llama.cpp b10969) and 35.0 tok/s (stock mlx-lm) to 48.3 tok/s (95% CI 46.2–50.6), all measured in one session: 2.1× and 1.38×, with byte-identical output on all prompts. The same code gives 1.32× on Qwen3.5-4B and 1.24× on Qwen3.5-2B. Accuracy is that of the MLX 4-bit model, which on MATH trails llama.cpp's Q4_K_M (81% vs. 87% at 4,096 tokens). We also report what did not help.

1 Introduction

Running an LLM locally is attractive for privacy and cost, but decoding is slow because every token streams all the weights from memory. A 4-bit Qwen3.5-9B reads about 4.5 GB of weights per token, so an M1 Pro with 200 GB/s of bandwidth can produce at most about 44 tok/s from weight reads alone. Speculative decoding [26, 6] works around this limit by guessing several tokens cheaply and verifying them together. The method rests on one assumption: verifying T tokens must cost about as much as producing one.

We find that this assumption is false for 4-bit models on an M1-class GPU, in both engines people use there. In MLX 0.32.4 the quantized mat-vec processes a T-token batch as T separate passes over the weights; MLX's own multi-vector kernel exists but is enabled for the affine format only on newer GPUs [23]. In llama.cpp (Ollama's bundled build, b10969) small-batch processing scales the same way, consistent with a recent consumer-hardware study [9]. With a 0.8B draft model, speculative decoding therefore runs at 0.63–0.96× of plain decoding. Qwen3.5 adds a second obstacle: 24 of its 32 layers are Gated DeltaNet recurrences [50], whose state cannot be truncated after a rejected draft.

Contributions. We present a measured, end-to-end case study and a working decoder:

- **Diagnosis.** Verification cost on the M1 Pro grows ≈linearly in T in both engines, and stock engine tuning has no effect (section 3).
- **A multi-token 4-bit verification kernel** for M1-class GPUs that reads each weight once and amortizes dequantization over $T \leq 4$ tokens. In its bit-exact form, two-token verification costs 1.23–1.47× a single-token mat-vec, against 1.56–1.83× for the path MLX dispatches (section 4.2). This is the same weight-reuse idea as MLX's `qmv_wide` and llama.cpp's few-row kernels, applied where those do not run (MLX) or postdate our baseline (llama.cpp).
- **Bit-exact verification.** The kernel reproduces MLX's single-token summation order row by row. A raw-activation identity keeps it in registers at T=3. Speculative output is then byte-identical to plain greedy decoding, not only equal in distribution, at 1–2% cost (section 4.3). To our knowledge this is the first fast multi-token quantized verifier with this property.
- **A complete self-speculative decoder.** It combines native MTP drafting [17, 12], standard snapshot rollback for linear-attention state, a trimmed draft vocabulary [56], compiled capture layers, adaptive draft length, and optimistic pipelining. For pipelining we identify an MLX-specific hazard: in-place KV writes from a discarded lazy graph still execute unless the cache views are pinned. Each component is ablated (section 5).
- **Where the remaining time goes.** On this GPU every *dependent* kernel costs about 10 µs of latency, while independent kernels overlap. Fusing each DeltaNet layer's dozen-kernel middle into one bit-exact Metal kernel adds 3.2% to plain and 1.9% to speculative decoding (section 4.7).
- **Negative results** that save others time (section 6).

2 Background and setup

Model. Qwen3.5-9B [40] has 32 decoder layers in a repeating pattern of three Gated DeltaNet (GDN) layers and one gated full-attention layer [38]. It uses a 4096-dimensional residual stream, 12,288-dimensional MLPs and a 248,320-token vocabulary, and ships a one-layer MTP head that predicts token t+2 from the final hidden state at t and the embedding of token t+1 [17, 12]. We use the MLX 4-bit conversion (affine quantization, group size 64; about 4.47 GB read per token) and Ollama's Q4_K_M GGUF (about 5.4 GB read per token).

Hardware and methodology. All measurements are on one Apple M1 Pro laptop with 16 GB of memory, macOS 14.7, MLX 0.32.4, mlx-lm 0.32.2, and llama.cpp b10969 (the `llama-server` bundled with Ollama 0.34.2).

- Decoding is greedy, single stream, with thinking disabled unless stated.
- tok/s means decode throughput after prefill.
- Before each configuration we wait for the macOS thermal state to be *nominal*.
- Because absolute speed drifts by up to ∼8% over hours of sustained load, every comparison we draw is made inside one session with configurations interleaved in rotating order.
- Low Power Mode was *on* by default on this machine and changed MLX results (section 3); unless noted, numbers are with it off.

Configuration (Qwen3.5-9B, 4-bit)	decode tok/s	prefill tok/s
llama.cpp b10969 (Ollama), Q4_K_M	22.35	233
+ any flag tuning (FA, threads, q8 KV, load mode)	22.0–23.0	218–234
+ native MTP drafting (n=3, 75–87% acceptance)	27.05	208
newer llama.cpp (b12068, few-row kernels)	23.29	238
+ native MTP drafting (n=3)	34.65	226
MLX 4-bit, bf16 parameters (as shipped)	34.82	170
MLX 4-bit, fp16 parameters	34.79	221
(Low Power Mode on: bf16 / fp16)	32.40 / 35.45	162 / 226

Table 1: Baselines on the M1 Pro (median of four prompts, 256 tokens). The M1 has no native bf16 arithmetic: with Low Power Mode on, fp16 parameters are 9% faster to decode; with it off, decode speeds are equal, but fp16 still raises prefill by 30%.

T	1	2	3	4	5	8
MLX `quantized_matmul`, MLP up (ms)	0.148	0.269	0.390	0.509	0.645	1.242
relative to T=1	1.00	1.82	2.64	3.44	4.36	8.39

Table 2: Cost of one 4-bit mat-vec ($4096 \to 12288$) with T input vectors on the M1 Pro, MLX 0.32.4.

3 Diagnosis

Stock engines. Table 1 lists the baselines. None of llama.cpp's options moves its speed: flash attention on or off, thread count, q8 KV cache, mmap, mlock or no-mmap all give 22.0–23.0 tok/s, with byte-identical output. MLX reaches 34.8 tok/s, i.e. 156 GB/s of weight reads at 4.47 GB/token, or 78% of the 200 GB/s peak. Its per-kernel mat-vec streams at 190–204 GB/s, while llama.cpp averages about 113 GB/s.

Verification cost grows with T. Table 2 times one 4-bit mat-vec with T input vectors. Timing 40 independent calls per evaluation removes launch overhead. MLX's cost grows almost exactly linearly up to T=6. From $T \geq 7$ it switches to a tiled kernel that costs a flat $\approx$8.6$\times$ a single vector. For the full forward pass, T=5 costs 2.9$\times$ T=1 when timed layer by layer: GDN layers 2.8$\times$, attention layers 2.9$\times$, LM head 4.0$\times$. In llama.cpp, processing 2 and 3 tokens costs 1.40$\times$ and 2.13$\times$. Raw compute is not the limit: the same GPU sustains 4.7 TFLOPS on fp16 GEMM, even in Low Power Mode. The kernels simply re-read the weights for each vector.

Consequence. With Qwen3.5-0.8B as the draft (44–81% acceptance), speculative decoding in MLX ran at 0.63–0.96$\times$ of plain decoding, and the output was identical. With the model's own MTP head and stock kernels it reached 1.10–1.27$\times$ at k=1 and fell below 1$\times$ for prose at $k \geq 2$ (table 5). llama.cpp's built-in MTP reached 27.05 tok/s at n=3 but was *slower* than plain decoding at n=1 and n=2, despite 84–95% acceptance. The newer release b12068 adds few-row kernels that make small batches cheaper. With them, the same MTP configuration reaches 34.65 tok/s, 1.49$\times$ its plain decoding, which supports the diagnosis: on this GPU, speculation pays once verifying several tokens is cheap. Its output differed from plain greedy decoding on 1 of the 4 prompts.

4 Method

4.1 MTP self-speculation with exact recurrent rollback

Each cycle drafts k tokens with the MTP head and verifies $[y, d_1, \ldots, d_k]$ in one target forward pass; accepted tokens and the target's correction token are committed.

- **Draft pass.** A catch-up pass writes the newly committed positions into the MTP layer's own KV cache, using the target's hidden states, and also yields d_1.
- **Chained drafts.** Further drafts feed the MTP's own output state back in.
- **Rolling back GDN state.** A GDN layer's (conv, ssm) state cannot be truncated, so during verification we run the GDN projections batched over all T positions but step the cheap recurrence one position at a time. That records the state after every position. Rollback restores the snapshot at the last accepted position and trims the attention KV caches. This is the standard technique [16, 1]; MLX's immutable arrays make the snapshots free references.

mlx-lm 0.32.2 refuses speculative decoding for this model because its recurrent cache is not trimmable.

One pitfall cost us hours. The checkpoint stores Qwen3.5's zero-centred RMSNorm weights, which must be loaded as $1+w$. The two MTP input norms, `pre_fc_norm_embedding` and `pre_fc_norm_hidden`, do not end in `norm.weight`, so a suffix rule skips them, and the drafts then come out as noise with <3% acceptance. Shifted correctly, the 4-bit MTP head predicts token $t+2$ 79% (GSM8K text) and 88% (MATH text) of the time under teacher forcing.

4.2 A multi-token verification kernel

We wrote the verification mat-vec as a Metal kernel (`mx.fast.metal_kernel`) for MLX's affine 4-bit format. Its layout follows MLX's single-token `qmv_fast`:

- two simdgroups per threadgroup and four output rows per simdgroup;
- each lane handles 16 consecutive weights per 512-weight block;
- results are reduced with `simd_sum`.

The difference is that each loaded and masked weight group is applied to all T activation vectors, which are held in registers. The kernel therefore streams the weights once and pays for dequantization once per weight rather than once per weight per token.

Several designs did not work.

- **simdgroup-matrix (8×8 MMA) kernels.** These cost the same for every $T \leq 8$, but in their best form 3.6× a single-vector pass. Staging through threadgroup memory or keeping fragments in registers gave the same result, and prefetching made it worse.
- **Scalar kernels without care for registers.** These spilled at $T \geq 3$ and ran up to 30× slower.

Unrolling, plus fp16 activation registers at $T \geq 3$, avoided the spills.

Table 3 and fig. 1 give the result. Our kernel is the fastest option at every T. At $T=2$ it costs 1.23–1.47× a single-token mat-vec, against 1.56–1.83× for the path MLX dispatches and 1.96–2.29× for MLX's own `qmv_wide`, and it is bit-exact. In an earlier session, with fp16 activation registers at $T=2$ (not bit-exact), it reached 0.93–1.25×. The absolute ratios move between sessions as the single-token baseline drifts, so we only compare kernels measured together.

matrix	kernel	T=2	T=3	T=4
GDN in-proj 4096×8192	MLX (dispatched)	1.83	2.57	3.42
	MLX `qmv_wide` (port, best k_{lanes}=4)	2.24	4.40	4.87
	ours	**1.47***	**1.99***	**2.27**
MLP up 4096×12288	MLX (dispatched)	1.73	2.54	3.31
	MLX `qmv_wide` (port)	2.16	4.23	4.77
	ours	**1.36***	**1.91***	**2.17**
MLP down 12288×4096	MLX (dispatched)	1.66	2.47	3.23
	MLX `qmv_wide` (port)	2.29	4.56	5.07
	ours	**1.32***	**1.85***	**2.12**
LM head 4096×248320	MLX (dispatched)	1.56	2.34	3.11
	MLX `qmv_wide` (port)	1.96	3.97	4.47
	ours	**1.23***	**1.71***	**1.98**

Table 3: Cost of a T-token verification mat-vec, relative to MLX's single-token kernel on the same matrix (one session; Low Power Mode off; fp16 parameters; each cell is the median of 7 runs of 40 queued calls). *Bit-identical, row by row, to MLX's single-token kernel. MLX's `qmv_wide`, run directly via a port of its source, is *slower* than MLX's fallback on this GPU. This is consistent with MLX enabling its affine path only on GPU generation 15 and later.

4.3 Bit-exact verification

In exact arithmetic, greedy speculative decoding returns the target's own argmax sequence. In practice the verify pass runs a different kernel than single-token decoding, so ties within rounding distance can flip.

- In a 1,500-token reasoning trace we observed such a flip at token 19: the two top logits were 27.297 and 27.281.
- Production systems accept this; vLLM documents speculation as lossless "up to the precision limits of hardware numerics" [43], and one study measured 95% exact-match [55].

We remove the discrepancy by construction.

Matching the summation order. MLX's single-token kernel computes, per lane and per 16-bit pack p of four nibbles $n_0..n_3$,

$$\text{acc} \mathrel{+}= \tilde{x}_0(p \,\&\, \texttt{0x000f}) + \tilde{x}_1(p \,\&\, \texttt{0x00f0}) + \tilde{x}_2(p \,\&\, \texttt{0x0f00}) + \tilde{x}_3(p \,\&\, \texttt{0xf000}),$$

with pre-scaled activations $\tilde{x}_j = x_j/16^j$. Each lane's group result is $s \cdot \text{acc} + b \cdot \sum x$, and the lanes are combined with `simd_sum`. Our kernel keeps the same lane-to-weight mapping, block size and expression association for every one of its T rows. Each output row is then bit-identical to MLX's single-token result, which we verified with maximum absolute difference exactly 0 on all four matrix shapes.

The raw-activation identity. Pre-scaled fp32 activations do not fit in registers at $T \geq 3$. Instead we keep the raw fp16 activations, which are exact, and extract nibbles by shifting. In fp32, $(x/16^j) \cdot (16^j n)$ and $x \cdot n$ are the same real number, rounded once, so they are bitwise equal: division by a power of two is exact and there is no underflow. The kernel is therefore still bit-exact at T=3

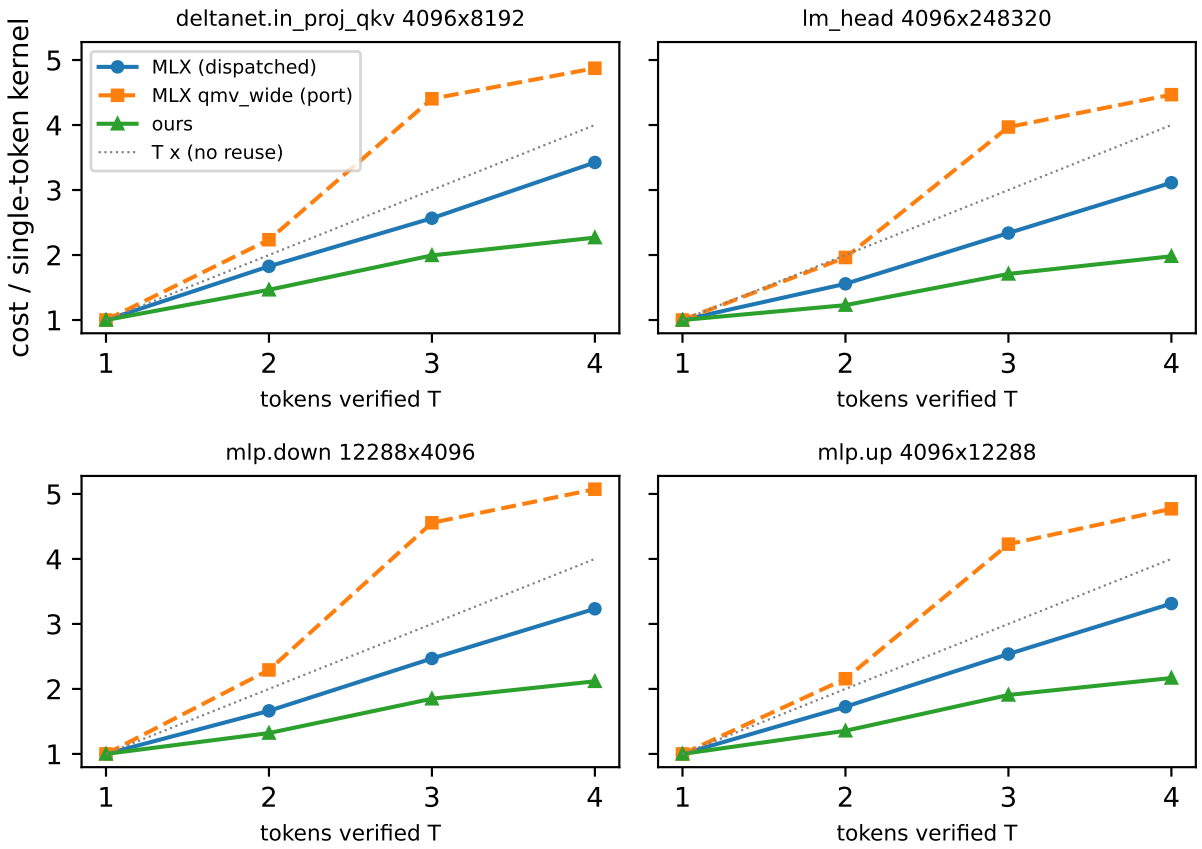


Figure 1: Cost of verifying T tokens with one 4-bit mat-vec, relative to MLX's single-token kernel, on four weight matrices of Qwen3.5-9B (the data of table 3). The dotted line is $T\times$, i.e. no weight reuse. Our kernel is bit-exact at $T\leq 3$; its $T=4$ point is the inexact fast-mode configuration.

and $T=4$. At $T=4$ it spills registers again, so our default exact mode drafts at most two tokens. A "fast" mode with up to three drafts uses an inexact $T=4$ path.

Cost. In an interleaved A/B on four prompts, exact verification ran at 49.9/50.7 tok/s against 50.9/51.2 for the inexact variant, a 1–2% cost. Exact mode reproduced plain greedy MLX output byte for byte on all prompts we tested: 4 short prompts, 3 600-token reasoning traces, and the 20-prompt suite on all three model sizes.

4.4 Trimmed draft vocabulary

The 248k-token head is about 80% of the cost of an MTP draft step. We draft over the 43,804 token ids observed in a corpus of English prose (WikiText), model-generated math solutions, and Python source. The target verifies with the full head, so a draft that needs a rarer token is simply rejected. This applies FR-Spec [56] to a native MTP head. A 32k cut lost acceptance on code and prose; 43.8k recovered it.

4.5 Optimistic pipelining

Each cycle needs one host synchronization to count accepted drafts. Building the next cycle's lazy graph took 2.25 ms of a 35.6 ms k=1 cycle (after compiling the capture-mode GDN layers; 4.3 ms before), and the GPU sat idle during it.

While cycle i is still verifying, we build cycle i+1 assuming all of its drafts will be accepted. That is the common case, and it needs no rewind: the caches are already in the post-verify state, and the bonus token, drafts and hidden states are lazy arrays. If the guess is right, the prebuilt graph launches immediately. If it is wrong, it is discarded before it runs, and the cache references are restored. This is the single-outcome case of overlapping speculation [32, 25].

The hazard. MLX implements KV writes as in-place updates that rebind the cache array to a new graph node. Resetting the cache offset therefore does *not* undo a discarded prebuild, and the next real step silently executes the whole discarded forward pass. This took the adaptive mode from 52.4 to 42.6 tok/s on a math prompt. The fix is to pin a zero-copy view (`c.keys[...]`) of each KV buffer before prebuilding and restore that view on a miss. With the fix, pipelining adds 7–10%.

4.6 Adaptive draft length

- After a fully accepted draft, k grows by one; after a rejection at position i, k becomes i+1.
- A running average of first-draft acceptance caps k: 1 below 0.8, 2 below 0.9, otherwise $k_{\max}$.
- $k_{\max}$=4 is always worse, because a 5-token verification falls back to MLX's slow path.

4.7 One kernel for the DeltaNet middle

At 4.46 GB of weights per token and the 190–204 GB/s that the mat-vec kernels reach on this GPU, weight streaming alone allows 42–46 tok/s, yet plain MLX decoding runs at 33–35. To locate the gap we timed the 9B forward pass with everything except its matrix products removed. The products alone, chained in layer order, take 25.6 ms per token (174 GB/s); the full pass takes 31.1 ms. The 5.5 ms remainder is the small operations between the products. Timed one by one, a single elementwise or normalization kernel on an 8–24 KB vector costs 9–14 µs, far more than its memory traffic. The cost is latency, not work: on this GPU every kernel that depends on its predecessor pays for a pipeline drain and refill. Independent kernels overlap, which is why fusing projections that share an input does nothing (Section 6).

A DeltaNet layer's middle is the longest such chain: depthwise convolution, SiLU, RMS normalization of q and k, the gate and β computations, the recurrence and the gated RMSNorm. That is about a dozen dependent kernels and 110–125 µs per layer in isolation, or 2.7–3.0 ms per token over 24 layers. We replace it with one Metal kernel per layer. Each threadgroup (1,024 threads) handles one value head; the head's 128×128 state stays in registers across the T positions of a verification, and the kernel writes the per-position snapshots that rollback needs.

Decoding stays bit-exact because every step copies the arithmetic of the MLX operation it replaces. That means the same intermediate dtypes, MLX's own formulas for sigmoid, softplus and `log1p`, the RMSNorm reduction order, and the gated-delta kernel's lane layout and butterfly reduction. A test compares all 24 DeltaNet layers of the 9B model at T=1–4 bit for bit against the unfused path, including the snapshots.

In isolation the fused middle takes 56–71 µs per layer, against 110–125 for the unfused chain. We measured it end to end in a paired A/B on the 20-prompt suite: stock and fused ran back to

M1 Pro, 16 GB	tok/s	95% CI	math	code	prose	chat
Qwen3.5-9B						
llama.cpp b10969 (Ollama)	23.09	[23.06, 23.12]	23.12	23.03	23.11	23.12
+ native MTP (n=3)	25.32	[23.69, 26.97]	30.34	26.21	21.44	23.28
stock mlx-lm	35.04	[34.89, 35.18]	34.98	35.03	34.90	35.23
ours, exact ($k\leq2$)	48.34	[46.15, 50.57]	55.36	47.03	**46.72**	**44.23**
ours, fast ($k\leq3$)	**49.23**	[46.69, 51.96]	**58.15**	**48.43**	46.50	43.84
Qwen3.5-4B						
llama.cpp b10969 (Ollama)	40.48	[40.42, 40.52]	40.41	40.44	40.53	40.53
stock mlx-lm	60.82	[60.60, 61.02]	60.44	60.88	60.83	61.12
ours, exact ($k\leq2$)	80.35	[76.27, 84.36]	92.53	**81.86**	75.25	**71.77**
ours, fast ($k\leq3$)	**80.49**	[76.21, 84.81]	**94.02**	81.11	**75.26**	71.57
Qwen3.5-2B						
llama.cpp b10969 (Ollama)	65.68	[64.23, 67.08]	69.56	67.55	63.96	61.65
stock mlx-lm	123.54	[123.30, 123.84]	123.01	123.46	123.54	124.14
ours, exact ($k\leq2$)	153.10	[146.78, 159.75]	173.66	152.37	144.87	141.49
ours, fast ($k\leq3$)	**155.65**	[148.04, 163.97]	**182.24**	**152.87**	**145.41**	**142.08**

Table 4: Mean decode throughput on the 20-prompt suite (thinking off, 256 new tokens, greedy, 3 rotating rounds, thermal state nominal throughout). "Ours" includes the fused DeltaNet kernel. Exact mode is byte-identical to plain greedy decoding on 20/20 prompts for each model. The 4B and 2B models have no llama.cpp MTP row: no MTP GGUF exists for them.

back on every prompt, in alternating order, for two rounds. Plain decoding speeds up 3.2% (95% CI 2.9–3.5%) and exact speculative decoding 1.9% (1.6–2.3%). Plain greedy output was identical to the unfused path in 40/40 runs.

A pitfall. In MLX, indexing an array with a Python integer (`x[0]`) is a gather, i.e. a copy, while `x[0:1]` and reshapes are free views. Our first version passed `cache[1][0]`, the 2 MB state, to the kernel. That copy added about 50 µs per layer and made the fused kernel slower than the chain it replaced. The decoder's hot path had three more such copies, and removing them is free.

5 Evaluation

Main result. Table 4 uses a fixed suite of 20 prompts: five math problems from held-out GSM8K/MATH test items, and five each of code, prose and chat. Each configuration runs three times, with the configuration order rotated between rounds. We report the mean of per-prompt means and a bootstrap 95% CI over prompts. All rows for one model come from one session, and "ours" includes the fused DeltaNet kernel (section 4.7). Exact mode produced byte-identical output to plain greedy decoding on 20/20 prompts for every model. Allowing a third draft (fast mode) adds 0.2–1.8%, well inside the confidence intervals, and gives up exactness. The same code runs the 4B and 2B models unchanged, with their own MTP heads (teacher-forced draft accuracy 0.88 and 0.85). The gain over stock mlx-lm shrinks with model size: 1.38× (9B), 1.32× (4B), 1.24× (2B). A smaller model streams its weights faster, so the per-cycle costs that do not scale with model size (graph construction, chains of small dependent kernels, the host sync) take a larger share of each cycle. For the same reason the fused DeltaNet kernel helps small models most: in an earlier session without it, the gains were 1.36×, 1.26× and 1.14×.

Configuration (9B)	median tok/s	note
llama.cpp b10969	22.4	
stock MLX (fp16)	34.8	bandwidth plateau
+ MTP self-speculation, k=1, MLX kernels	40.2	lossless
+ multi-token verification kernel	43.8	full draft head
+ trimmed 32k draft head	46.6	
+ 43.8k draft head, adaptive k	49.1	
+ compiled capture-mode GDN layers	50.2	
+ tuned kernel registers	52.2	
+ optimistic pipelining	55.6	peak 64.7 (math)
One later session (plain MLX 33.6–33.8):		
full system, fast ($k\leq3$)	51.1	
full system, exact ($k\leq2$)	51.1	byte-identical to stock
+ fused DeltaNet middle (section 4.7), exact	**52.5**	byte-identical; plain 35.2
+ fused DeltaNet middle, fast	52.1	

Table 5: Cumulative ablation on Qwen3.5-9B, M1 Pro, thinking off, 256 tokens.

Ablation. Table 5 adds the components one at a time. All rows are medians of the same four prompts at 256 tokens; within-session differences are meaningful, but absolute numbers drift between sessions. The upper rows were measured as the system was built; the bottom block remeasures the final configurations together in one session, 8% below the earlier peak. In that session exact and fast mode run at the same speed, so exactness costs nothing measurable there.

Long reasoning traces. With thinking on and 1,500 generated tokens on three prompts (word problem, LRU-cache code, history essay), the mean throughput was 22.9 tok/s for llama.cpp, 33.0 for stock MLX and 50.5 for our decoder ($k\leq3$): 2.2× and 1.53×. Draft acceptance was 71–81% and the thermal state stayed nominal.

Quality. Exact mode is byte-identical to stock greedy MLX, so its accuracy is that of the MLX 4-bit model. On the same 100 GSM8K-train and 100 MATH problems (2,048-token cap) used to score the Q4_K_M baseline (GSM8K 95%, MATH 76%), exact mode scores 93% on GSM8K (0 gained, 2 lost; p=0.50) and 71% on MATH at the 2,048-token cap. For comparison, mlx-lm's batched generator produces the same text as single-stream greedy decoding for only 84 of the 100 GSM8K problems: batched numerics differ, which is why exactness has to be engineered. The MATH difference is 2 problems gained against 7 lost, which is not significant (exact McNemar p=0.18). All 7 losses are answers cut off at the cap: the MLX 4-bit model writes 14% longer solutions than the Q4_K_M model (1,218 vs. 1,072 tokens on average) and hits the cap 28 times vs. 22. The speculative decoder does not cause any of this, since its output is byte-identical to stock MLX; it is a property of the two 4-bit quantizations. At a 4,096-token cap the gap remains: 81% vs. 87% on MATH (1 gained, 7 lost; p=0.07) and 94% vs. 95% on GSM8K. Pooled over both sets, 1 gained against 8 lost is significant (p=0.04). All 7 MATH losses are again unfinished answers: the MLX model writes 18% longer solutions (1,635 vs. 1,380 tokens) and does not finish within 4,096 tokens 17 times vs. 12. Moving from llama.cpp's Q4_K_M file to the MLX 4-bit model therefore costs some long-form math accuracy; our decoder changes nothing further.

6 What did not help

- **llama.cpp flags and load modes:** no effect (22.0–23.0 tok/s).
- **Low Power Mode:** no effect on llama.cpp; −7% for MLX with bf16 parameters, and no effect with fp16 parameters.
- **Projection fusion** (concatenating q/k/v, gate/up, and the four GDN input projections, which is exact for row-quantized weights): +0.1% plain, ≈0 speculative. These projections share an input, so their kernels already overlap; only chains of dependent kernels cost latency (Section 4.7).
- **Lossless weight compression:** the empirical entropy of the 4-bit codes is 3.71 bits, so even ideal entropy coding would remove at most about 6.5% of the bytes per token, before paying for decoding on the GPU.
- **Wiring the model in memory** (`mx.set_wired_limit`, as `mlx_lm.generate` does): no gain (plain 31.1 wired vs. 32.1 tok/s unwired; exact speculative 48.2 vs. 49.2).
- **Prefetching the next layer's weights** into the system-level cache with a concurrent read: 2.7 ms per token slower.
- **Compiling the single-token GDN step:** +1.8% synchronous, ≈0 with asynchronous evaluation.
- **simdgroup-matrix verification kernels:** flat in T but ≥3.6× a single-token pass.
- **Tree drafting:** on our cost curve, a 3-token tree verification costs 1.25–1.8× while adding $\lesssim$0.15 expected tokens per cycle.
- $k_{\max}$=4**:** 5-token verification leaves the fast kernel, so it is slower everywhere.
- **A 0.8B draft model:** the draft step costs 8 ms, too large relative to a 30 ms target step on this GPU.

7 Related Work

Speculative decoding. Draft-then-verify decoding goes back to blockwise parallel decoding [41]. Speculative decoding and speculative sampling [26, 6] added an acceptance rule that preserves the target distribution, so under greedy decoding the output equals the target's own argmax sequence, in exact arithmetic. Later work improves the drafter: extra decoding heads (Medusa [5]), feature-level drafters (EAGLE [28, 27, 29]), a recurrent drafter (ReDrafter [8]), layer skipping [54], Jacobi-style n-gram guessing [14], and drafters that emit a whole block in one pass (DFlash [7], DFlare [53], Liquid AI's DSpark [31]). Others verify token trees instead of chains [34, 27, 49]. FR-Spec [56] restricts the draft LM head to a frequency-ranked vocabulary subset, and the released EAGLE-3 checkpoints draft over 32k of 128k tokens; our 43.8k-token MTP head applies this idea to Qwen3.5's 248k vocabulary and is not new. Adaptive draft length (SpecDec++ [22], PEARL [32]) is likewise established, as is overlapping drafting with verification: PEARL drafts during verification, and Speculative Speculative Decoding [25] prepares speculations for predicted verification outcomes. Our optimistic pipelining is the single-outcome case of that idea (predict full acceptance) on one device. What we add there is an MLX-specific hazard: in-place KV-cache writes inside a discarded

lazy graph still execute unless the KV views are pinned. Our negative result for tree drafting reflects our measured verification cost curve; tree and block methods assume verification of many tokens is nearly free, which it is not on an M1 Pro.

Multi-token prediction. Gloeckle et al. [17] train several output heads on a shared trunk and report up to 3× faster inference by using them for self-speculation. DeepSeek-V3 [12] trains sequential MTP modules and reuses them as a drafter, with 85–90% second-token acceptance and 1.8× tokens per second. Qwen3-Next [39] and Qwen3.5 [40] ship MTP modules intended for inference; Qwen3.5-9B stacks eight blocks of three Gated DeltaNet layers and one gated-attention layer [38]. MTP self-speculation for this family already exists in llama.cpp [3], in an open mlx-lm pull request [1], and in MTPLX [2]. We claim no novelty for MTP drafting. Our contribution is making its verification cheap and exact on hardware where, by default, it is neither.

Linear-attention and hybrid models. Gated DeltaNet [50] combines the delta rule of DeltaNet [51] with the gating of Mamba-2 [11, 19]. Its layers overwrite a fixed-size state, so rejected drafts cannot be dropped by truncating a cache. The Mamba in the Llama [44] gave a speculative decoding algorithm for distilled hybrids and STree [48] gave tree verification for SSMs. Work from 2026 targets Gated DeltaNet hybrids in GPU serving engines: TreeWY [16] and Bole [45] compute tree updates in closed form and rebuild only the accepted state, SpecLA [46] stores compact factors to recover it, Weaver [35] verifies trees without rollback, KVBuffer [57] defers state updates by buffering keys and values, LumoTree [33] (first released as "GDN Tree-Scan") replays branch-local scans inside vLLM, and ReplaySSM [10] caches inputs instead of states. TreeWY and ReplaySSM both describe snapshotting the recurrent state at every draft position as current practice [16, 10], and the mlx-lm pull request does the same [1]. Our rollback steps the recurrence per position inside the batched verify pass and keeps every (conv, SSM) state, so it is that standard technique, not a new one. It fits our setting because one sequence, chains of at most four tokens, and a 9B model make the snapshots small, whereas the systems above attack the memory and tree-width limits of batched GPU serving.

On-device and Apple Silicon. ReDrafter [8] reported up to 2.3× on Apple Silicon with an implementation in MLX [20]. Chordiya [9] found on an M3-class laptop that the quantized Metal backend of llama.cpp [15] verifies draft tokens serially, with verify cost growing nearly linearly in the number of drafts for small batches, and three of five configurations ran slower than plain decoding. We see the same effect in MLX 0.32.4 on an M1 Pro. MLX's small-batch kernel `qmv_wide` [23], adapted from llama.cpp's `mul_mv_ext`, is enabled for affine quantization only on GPU generation 15 (M3) and newer, so 4-bit verification on M1-class GPUs still runs one mat-vec per token. llama.cpp has since added few-row simdgroup-matrix kernels for speculative verification on Apple7+ GPUs [37] (build b11404), after the b10969 build we use as a baseline. Our multi-token kernel uses the same idea, reading each weight once for T tokens, for affine 4-bit weights on an M1 Pro; what sets it apart is the summation-order constraint described next. MTPLX reports 1.6× on a 16 GB M4 Mac mini and the mlx-lm pull request 1.57× for a 27B model on an M4 Pro; these figures use other chips and models and are not directly comparable to ours. Weight quantization (GPTQ [13], AWQ [30], llama.cpp k-quants [24]) and roofline analyses of decoding [47, 36, 52] explain why single-stream decoding is bandwidth-bound and why one weight read has to serve several tokens.

Exactness and determinism. Speculative decoding is lossless in exact arithmetic, but batched verification uses different kernels and reduction orders from single-token decoding, so greedy outputs

can flip at near-ties. vLLM states its guarantee only up to hardware numerical precision [43]. Zhang et al. [55] report 95% exact match with standard decoding and trace the remainder to floating-point non-determinism. An open llama.cpp issue [4] traces MTP greedy divergence on quantized targets to different single- and multi-token kernels, and restores bit-identity only by forcing the multi-token pass onto matching kernels. Batch-invariant kernels [21, 42] make results independent of batch size by fixing the reduction strategy, and LLM-42 [18] enforces determinism with a verify-and-rollback loop over fixed-shape reductions. Our kernel applies the batch-invariance principle to speculative verification on Metal. It reproduces, row by row, the summation order of MLX's single-token `qmv` while still reading each weight once, so speculative output is byte-identical to plain greedy decoding, not only equal in distribution. We found no prior work that makes a fast multi-token quantized verifier bit-identical to the single-token path; in our implementation the exactness costs 1–2%.

8 Limitations

- **One device.** All results come from one M1 Pro. Newer GPUs (M3 and later) enable MLX's `qmv_wide` and have more compute headroom, so the gain from our kernel will be smaller there; the bit-exact construction still applies.
- **Older llama.cpp.** Our llama.cpp baseline is Ollama's bundled b10969, which predates the few-row Metal kernels [37]. The newer release b12068, which has them, decodes only 4% faster without speculation (23.3 vs. 22.4 tok/s, identical output). With native MTP drafting it reaches 34.7 tok/s on the four-prompt set, against 52.5 for our exact mode on the same prompts measured in a separate session (about 1.5×). We did not run b12068 on the full 20-prompt suite.
- **Unbenchmarked alternative.** We did not run the open mlx-lm MTP pull request [1]: it needs the 19 GB checkpoint re-converted with its own code. It reports 1.57× for a 27B model on an M4 Pro, a newer GPU on which MLX's multi-vector kernel is enabled.
- **Greedy decoding only.** Exactness for sampling would also need matched sampling arithmetic.
- **Hand-tuned kernels.** The kernel configurations are tuned for this GPU.
- **Model coverage.** We evaluate Qwen3.5-9B, 4B and 2B. Larger models do not fit in 16 GB.

9 Conclusion

On an M1-class laptop GPU, speculative decoding stands or falls on whether verifying a few tokens costs about the same as producing one. In both mainstream engines it does not, for 4-bit models. A small multi-token kernel fixes this. Matching the single-token kernel's arithmetic makes speculation bit-exact at negligible cost. With native MTP drafting, the hybrid Qwen3.5-9B decodes 1.38× faster than stock mlx-lm (1.58× on math) and 2.1× faster than llama.cpp b10969, with byte-identical output. It is also about 1.5× faster than the newer llama.cpp b12068 running its own MTP drafting. Code and raw results: `https://github.com/ganisai8202/qwen35-bitexact-spec-mlx`.

Disclosure. AI tools were used to assist with writing the code and generating the figures.

References

[1] AirRunner. Native MTP speculative decoding (Qwen3.5/3.6 reference implementation). mlx-lm pull request #990 (open as of 2026-11-03), 2026.

[2] Youssof Altoukhi. MTPLX: native multi-token-prediction speculative decoding on Apple silicon. Software, 2026.

[3] am17an. llama + spec: MTP support. llama.cpp pull request #22673, merged 2026-05-16, 2026.

[4] Ankk98. Eval bug: Speculative decoding (draft-mtp / draft-dspark): greedy output diverges from vanilla on quantized targets. llama.cpp issue #25618 (open as of 2026-11-03), 2026.

[5] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[6] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[7] Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. In *International Conference on Machine Learning (ICML)*, 2026.

[8] Yunfei Cheng, Aonan Zhang, Xuanyu Zhang, Chong Wang, and Yi Wang. Recurrent drafter for fast speculative decoding in large language models. *arXiv preprint arXiv:2403.09919*, 2024.

[9] Param Chordiya. Lossless but not free: An empirical anatomy of speculative decoding on consumer hardware. *arXiv preprint arXiv:2607.17283*, 2026.

[10] Tri Dao. ReplaySSM: Cache SSM inputs, not state. Blog post, 2026.

[11] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024.

[12] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[13] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

[14] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of LLM inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*, 2024.

[15] Georgi Gerganov and ggml-org contributors. llama.cpp: LLM inference in C/C++. Software, build b10969 (as bundled with Ollama) used as baseline, 2023.

[16] Sneha Murthy Ghantasala. TreeWY: Speculative verification for gated DeltaNet hybrids. *arXiv preprint arXiv:2608.20961*, 2026.

[17] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.

[18] Raja Gond, Aditya K. Kamath, Ramachandran Ramjee, and Ashish Panwar. LLM-42: Enabling determinism in LLM inference with verified speculation. *arXiv preprint arXiv:2601.17768*, 2026.

[19] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[20] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. Software, version 0.32.4 used in this work, 2023.

[21] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, September 2025.

[22] Kaixuan Huang, Xudong Guo, and Mengdi Wang. SpecDec++: Boosting speculative decoding via adaptive candidate lengths. In *Conference on Language Modeling (COLM)*, 2025.

[23] jessegross. Add small-batch quantized matvec kernel (qmv_wide). MLX pull request #3764, merged 2026-06-26, 2026.

[24] Iwan Kawrakow. k-quants. llama.cpp pull request #1684, merged 2023-06-05, 2023.

[25] Tanishq Kumar, Tri Dao, and Avner May. Speculative speculative decoding. In *International Conference on Learning Representations (ICLR)*, 2026.

[26] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.

[27] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858*, 2024.

[28] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.

[29] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025.

[30] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Conference on Machine Learning and Systems (MLSys)*, 2024.

[31] Liquid AI. Up to 3.2x faster inference with LFM2.5-DSpark. Hugging Face blog, August 2026.

[32] Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. PEARL: Parallel speculative decoding with adaptive draft length. In *International Conference on Learning Representations (ICLR)*, 2025.

[33] Zhiyuan Ma. LumoTree: Path-parallel speculative verification for hybrid language models. *arXiv preprint arXiv:2609.23900*, 2026. v1 titled "GDN Tree-Scan: Served Tree Verification for Recurrent-Hybrid Language Models".

[34] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating generative large language model serving with tree-based speculative inference and verification. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[35] Yuma Oda, Ryan Mathieu, Roman Knyazhitskiy, and Artur Chakhvadze. Trees from marginals: Autoregressive drafting with factorized priors. *arXiv preprint arXiv:2607.06763*, 2026.

[36] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *arXiv preprint arXiv:2211.05102*, 2022.

[37] pratiknarola-t. metal : few-row MMA mat-mul. llama.cpp pull request #29869, merged 2026-10-05, release b11404, 2026.

[38] Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free. *arXiv preprint arXiv:2505.06708*, 2025.

[39] Qwen Team. Qwen3-Next-80B-A3B-Instruct model card. Hugging Face, 2025.

[40] Qwen Team. Qwen3.5: Towards native multimodal agents. Blog post; model card at https://huggingface.co/Qwen/Qwen3.5-9B, February 2026.

[41] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[42] vLLM Project. Batch invariance. vLLM documentation, 2026.

[43] vLLM Project. Speculative decoding. vLLM documentation, section "Lossless guarantees of Speculative Decoding", 2026.

[44] Junxiong Wang, Daniele Paliotta, Avner May, Alexander M. Rush, and Tri Dao. The Mamba in the Llama: Distilling and accelerating hybrid models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[45] Li Wang, Yi Su, Xiabao Wu, Chiran You, Yongchao Liu, Zhan Qiu, Juelu Zhang, Jiajun Zheng, Fangxin Liu, Jie Zhang, Chen Tian, and Chengying Huan. Bole: Efficient tree speculation for hybrid-attention language models. *arXiv preprint arXiv:2608.01651*, 2026.

[46] Zhibin Wang, Xuying Han, Zhaohua Yang, Fuliang Liu, Xue Li, Rong Gu, Sheng Zhong, and Chen Tian. SpecLA: Efficient speculative decoding for linear-attention models. *arXiv preprint arXiv:2607.16673*, 2026.

[47] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[48] Yangchao Wu, Zongyue Qin, Alex Wong, and Stefano Soatto. STree: Speculative tree decoding for hybrid state-space models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[49] Yunfan Xiong, Ruoyu Zhang, Yanzeng Li, Tianhao Wu, and Lei Zou. DySpec: Faster speculative decoding with dynamic token tree structure. *arXiv preprint arXiv:2410.11744*, 2024.

[50] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025.

[51] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024.

[52] Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Zhe Zhou, Chenhao Xue, Bingzhe Wu, Zhikai Li, Qingyi Gu, Yong Jae Lee, Yan Yan, Beidi Chen, Guangyu Sun, and Kurt Keutzer. LLM inference unveiled: Survey and roofline model insights. *arXiv preprint arXiv:2402.16363*, 2024.

[53] Jiebin Zhang, Zhenghan Yu, Song Liu, Eugene J. Yu, Zheng Li, Dawei Zhu, Jiangshan Duo, Weimin Xiong, Yifan Song, Guanghua Yu, Jianchen Zhu, and Sujian Li. DFlare: Scaling up draft capacity for block diffusion speculative decoding. *arXiv preprint arXiv:2606.02091*, 2026.

[54] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft & verify: Lossless large language model acceleration via self-speculative decoding. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[55] Ranran Haoran Zhang, Soumik Dey, Ashirbad Mishra, Hansi Wu, Binbin Li, and Rui Zhang. Correctness forensics for batch speculative decoding: Diagnosing the ragged tensor problem. In *Findings of the Association for Computational Linguistics: EMNLP*, 2026.

[56] Weilin Zhao, Tengyu Pan, Xu Han, Yudi Zhang, Ao Sun, Yuxiang Huang, Kaihuo Zhang, Weilun Zhao, Yuxuan Li, Jianyong Wang, Zhiyuan Liu, and Maosong Sun. FR-Spec: Accelerating large-vocabulary language models via frequency-ranked speculative sampling. *arXiv preprint arXiv:2502.14856*, 2025.

[57] Longwei Zou and Lin Zhong. KVBuffer: IO-aware serving for linear attention. *arXiv preprint arXiv:2605.19049*, 2026.